

1. Title

AgriSmart: Climate-Smart Agriculture Decision Support System

2. Abstract

AgriSmart is a web-based decision support application designed to help farmers and agriculture learners make informed crop and farm-management decisions. The system integrates weather-based advisories, crop recommendation logic, crop problem diagnosis, daily crop planning, and basic admin analytics in one platform.

The application is developed using React, TypeScript, Vite, Tailwind CSS, and client-side data/context architecture. It supports multilingual interaction (Marathi, Hindi, English) and role-based access (farmer, student, admin).

The project demonstrates a practical, scalable front-end architecture for smart agriculture use cases, while identifying future enhancements such as backend integration, secure authentication, and advanced predictive models.

3. Introduction

Agriculture faces uncertainty due to climate variability, pest outbreaks, water stress, and market fluctuation. Small and medium farmers often make decisions with limited real-time information. AgriSmart addresses this by offering a unified digital assistant for day-to-day agricultural decisions, including weather advisories, crop suitability checks, and crop health guidance.

4. Problem Statement

Farmers and students often lack:

A single platform for weather, crop planning, and problem diagnosis.

Localized guidance for crop selection and risk prevention.

Easy, multilingual, mobile-friendly decision support.

5. Objectives

Provide live weather insights with actionable advisories.

Recommend suitable crops based on user inputs (soil, season, climate range).

Help users identify crop problems and suggest treatment/prevention.

Generate daily crop care guidance and simple revenue estimation.

Support multilingual UX and role-based access.

6. Scope

Web application for educational and practical planning use.

Client-side operation with static crop/problem datasets.

Live weather via external APIs.

Admin-level dashboard analytics.

Future scope includes backend, secure auth, IoT integration, and predictive ML.

7. Development Methodology

Incremental modular development was used:

Base app setup and routing.

Authentication and language context.

Feature modules (dashboard, recommendation, solver, planner).

Admin analytics and UI improvements.

Build/testing/lint checks.

8. Feasibility Study

8.1 Technical Feasibility

Uses modern, stable frontend stack.

API integration is straightforward.

Runs in browser without heavy infrastructure.

8.2 Economic Feasibility

Open-source libraries reduce cost.

Low deployment cost on static hosting platforms.

8.3 Operational Feasibility

Simple UI and guided forms.

Language support improves adoption among local users.

9. Software and Tools

React 18

TypeScript

Vite

Tailwind CSS
React Router
TanStack Query
Recharts
Vitest + Testing Library
ESLint
10. System Requirements
10.1 Hardware
Standard laptop/desktop
4 GB RAM minimum
10.2 Software
Node.js + npm
Modern browser (Chrome/Edge/Firefox)
VS Code (recommended)
11. Requirement Analysis
11.1 Functional Requirements
User registration/login/logout
Role-based route access
Weather search by city and geolocation
Crop database listing/search
Crop recommendation engine
Crop problem diagnosis and report download
Daily planner with stage/profit estimation
Admin analytics dashboard
Language switching (EN/HI/MR)
11.2 Non-Functional Requirements
Responsive UI
Usability for non-technical users
Maintainable modular code
Basic performance for dashboard and search
Extensible architecture for future backend
12. System Architecture
AgriSmart follows a client-side layered architecture:

Presentation Layer: React pages + reusable UI components.
Application Layer: Context providers (Auth, Language) + hooks.
Service Layer: Weather service and advisory/scoring logic.
Data Layer: Static datasets and localStorage persistence.
13. Module Description
13.1 Authentication Module
Handles login/register/logout.
Stores session and users in localStorage.
Enforces protected and admin routes.
13.2 Language Module
Supports Marathi, Hindi, English.
Loads JSON translation dictionaries.
Persists selected language in localStorage.
13.3 Dashboard Module
Displays weather metrics and advisories.
Provides quick navigation to feature modules.
Includes crop weather score and disease warning summary.
13.4 Crop Database Module
Displays crop cards with agronomic attributes.
Search by crop name.
13.5 Crop Recommendation Module
Accepts soil/season/temp/rainfall inputs.
Scores crops by overlap and suitability.
Returns top recommendations with confidence and reasons.
13.6 Crop Problem Solver Module
Filters by crop and problem type.
Matches symptom keywords.
Shows treatment, prevention, recovery time.
Exports text report.
13.7 Daily Planner Module

Calculates growth stage from sowing date.
Generates irrigation/fertilizer/risk guidance.
Estimates yield and revenue by land size.

13.8 Admin Module

Displays user/crop/problem analytics via charts.
Shows registered user list and summary metrics.

14. DFD (Textual)

14.1 Level-0 DFD

External entities: User, Admin, Weather APIs

System: AgriSmart

Flows:

User inputs credentials/filters/query.
System returns recommendations, advisories, and plans.
System requests weather data from external APIs.
Admin receives analytics and user summaries.

14.2 Level-1 DFD

Auth Process: input credentials -> validate local user store -> create/remove session.

Weather Process: city/coordinates -> weather service -> advisory generation -> dashboard.

Recommendation Process: input constraints -> scoring engine -> ranked crops.

Problem Solver Process: filters + symptom text -> match dataset -> diagnosis output.

Planner Process: crop + sowing date + land size -> stage/risk/profit engine -> daily plan.

15. UML (Textual)

15.1 Use Case Diagram (Actors)

Farmer: login, weather check, recommendation, diagnosis, planner.

Student: same as farmer for learning/planning.

Admin: all user actions + admin analytics.

15.2 Class/Entity View

User: id, name, email, role, createdAt

Crop: agronomic parameters + yield + market price

CropProblem: type, explanation, treatment, prevention, confidence

WeatherData: temperature, humidity, wind, rainfall probability, condition

Services: weather fetch, advisory scoring, crop-weather score

15.3 Sequence (Recommendation)

User submits form.

UI validates inputs.

Recommendation logic filters/scorers run.

Top results rendered with reasons/confidence.

16. Implementation Highlights

Routing and access control implemented in app-level route guards.

Reusable dashboard layout and UI component library used for consistency.

Weather service has primary+fallback provider strategy.

Language translations structured via JSON files.

Data modules are static and easy to extend.

17. Testing and Validation

17.1 Testing Types

Functional checks of each page workflow.

Build validation.

Lint and unit-test command execution.

17.2 Current Validation Status

build: Pass

test: Fail due to setup file path mismatch

lint: Fails with some TypeScript/ESLint errors

17.3 Sample Test Cases

Test Case	Input	Expected Result	Current Result
Login Success	Valid demo credentials	Redirect to dashboard	Pass
Protected Route	Open /dashboard without login	Redirect to /login	Pass
Crop Search	Search "wheat"	Matching crop appears	Pass
Recommendation	Soil+season input	Top suggestions with score	Pass
Problem Solver	Crop + type + description	Relevant diagnosis cards	

Pass

Build npm run build Production bundle Pass

Unit Test npm run test Tests execute Fail (setup path)

18. Limitations

No backend database/API for persistent secure records.

Password/session handling is client-side only (not production secure).

Recommendation/planner modules use simplified rule logic.

Test setup configuration issue currently blocks automated test execution.

Large production JS bundle indicates optimization need.

19. Future Enhancements

Backend with secure authentication and database.

Real farm profile history and personalized advisory engine.

ML-based disease prediction and crop yield forecasting.

GIS/soil map integration.

Offline/PWA mode for rural low-connectivity areas.

IoT sensor integration (soil moisture, pH, temperature).

Market price API integration with region-wise recommendations.

20. Conclusion

AgriSmart successfully demonstrates a practical climate-smart agriculture support platform with modular architecture, multilingual UI, weather integration, and decision-support workflows. It is a strong prototype suitable for academic demonstration and MVP use. With backend security, testing stabilization, and advanced predictive enhancements, it can evolve into a production-ready agri advisory system.